

**Al-Baha University**  
Faculty of Engineering  
Computer Engineering Department

Software Engineering Course Project

---

# **Cloud Store**

Object-Oriented Analysis and Design

---

Prepared by  
**Kinan Alghamdi**

Individual Project

Table of Contents

1. Introduction . . . . . 2

2. Problem Statement . . . . . 2

    2.1 Core Problem . . . . . 2

    2.2 The Impact . . . . . 2

    2.3 The Solution . . . . . 2

3. Functional Requirements . . . . . 3

    3.1 Customer Transaction (UC2) and Payment (UC3) . . . . . 3

    3.2 Storefront Management and Inventory (UC1) . . . . . 3

    3.3 Order Fulfillment and Logistics (UC4) . . . . . 3

    3.4 Reporting and Analysis (UC5) . . . . . 4

4. Requirements Elicitation . . . . . 4

    4.1 Actors and Scenarios . . . . . 4

    4.2 Use Cases . . . . . 5

5. Functional Model . . . . . 5

    5.1 Main System Scenario: Seamless Purchase . . . . . 5

    5.2 Detailed Scenario: Complete Digital Grocery Purchase and Delivery . . . . . 7

6. Non-Functional Requirements . . . . . 8

7. Analysis Object Model . . . . . 8

    7.1 Class Diagram . . . . . 9

8. Dynamic Model . . . . . 10

    8.1 Sequence Diagram: Place Order and Reserve Stock (UC2) . . . . . 10

    8.2 Sequence Diagram: Control Order Fulfillment Cycle (UC4) . . . . . 10

    8.3 State Machine: FulfillmentManager Control Object . . . . . 11

9. Business Process Models . . . . . 12

    9.1 Order Processing and Fulfillment . . . . . 12

    9.2 Inventory Restock Management . . . . . 13

# 1. Introduction

The Cloud Store project is a greenfield development aiming to create a highly scalable platform that unifies global e-commerce with integrated logistics. The primary objective is to resolve market fragmentation by ensuring reliable purchase, payment, and home delivery worldwide.

The core objectives were:

1. To generate a complete **Functional Model** detailing all system capabilities (e.g. Place Order, Control Order Fulfillment).
2. To construct a stable **Object Model** by defining the structure (entity objects) and behavior (sequence diagrams) of the application domain.
3. To establish stringent **Non-Functional Requirements** for scalability, security, and availability.

These phases yielded the required OOSE outcomes: a complete requirements specification, a visual use case model defining scope, a conceptual data structure, and a protocol for object interactions. Together they serve as the blueprint for system design, ensuring the Cloud Store is a robust and efficient solution.

## 2. Problem Statement

### 2.1 Core Problem

The retail grocery and delivery market lacks true global accessibility and integration, preventing customers from conveniently and reliably purchasing a wide range of goods from anywhere in the world for direct home delivery. Existing solutions are often restricted by geographic boundaries, lack robust unified logistics management, or fail to provide transparent, real-time status updates from order placement through final delivery.

### 2.2 The Impact

This fragmentation results in poor customer experience due to inconsistent delivery windows, lack of stock visibility, and opaque tracking. For businesses, it leads to inefficient logistics, difficulty scaling inventory management globally, and a lack of consolidated performance data needed for investor decision-making.

### 2.3 The Solution

Cloud Store solves this with a single, highly available, object-oriented application that unifies the digital shopping experience with resilient, integrated fulfillment logistics. The platform provides seamless purchasing and payment while giving managers advanced tools for inventory control and optimized route generation, resolving the primary pain points for customers, operations, and investors alike.

### **3. Functional Requirements**

#### **3.1 Customer Transaction (UC2) and Payment (UC3)**

1. The system shall allow a registered customer to browse and search the product catalog and add selected items to their shopping cart.
2. The system shall allow the customer to modify the contents and quantities within their cart prior to checkout.
3. The system shall compute the total cost of the order, including applicable taxes and delivery fees.
4. The system shall allow the customer to submit the order and initiate the payment process using a saved or newly entered payment method.
5. The system shall securely interface with the Payment Service to transmit transaction details and receive a definitive Success or Failure authorization response.
6. The system shall lock the order status as Paid only upon receipt of a Success response from the Payment Service.

#### **3.2 Storefront Management and Inventory (UC1)**

7. The system shall provide an interface for the Store Owner/Manager to add, edit, or deactivate products, including setting price and descriptive attributes.
8. The system shall maintain a real-time count of all product stock levels and their corresponding warehouse locations.
9. The system shall automatically generate alerts when the stock level for any product drops below a configurable safety minimum.
10. The system shall allow the Store Owner/Manager to generate and print optimized Purchase Orders (POs) for suppliers based on current stock, sales history, and demand forecast.

#### **3.3 Order Fulfillment and Logistics (UC4)**

11. The system shall generate optimized picking lists for warehouse staff based on the location of items within a batch of orders.
12. The system shall reserve (or decrement) inventory stock immediately upon the start of the physical picking process.
13. The system shall use an algorithm to generate an optimized delivery route for the Delivery Worker based on customer delivery time windows and geographical locations.
14. The system shall allow the Delivery Worker (via the mobile app) to view their assigned route and update the status of individual orders (e.g. Out for Delivery).

15. The system shall capture and store proof of delivery (digital signature or timestamped photo) for every completed order.

### 3.4 Reporting and Analysis (UC5)

16. The system shall provide the Investor with a secure, read-only dashboard for viewing key business metrics.
17. The system shall generate standardized reports displaying aggregated metrics such as GMV, Customer Acquisition Cost (CAC), and Customer Retention Rate.
18. The system shall allow the export of historical sales and inventory data for external analysis and auditing.

## 4. Requirements Elicitation

The actors were selected to model every essential participant in the system's value chain. Customers, Store Owners/Managers, and Delivery Workers form the core transactional and logistical triad, driving revenue and physical fulfillment. The Payment Service was included as a mandatory integrator to handle financial transactions securely at the system boundary, and Investors were included to model the system's passive reporting requirements.

Scenarios were derived from the most critical business processes: the order processing and fulfillment chain (the primary revenue driver), inventory and purchase order management (the core support mechanism), and the mandatory payment integration. This kept all UML and BPMN modeling focused on the high-value, high-complexity interactions needed to launch the product.

### 4.1 Actors and Scenarios

| Actor               | Scenario                                                                                                                                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer            | Logs in, chooses products, adds them to the shopping cart, completes the order, and pays. The store then prepares the order and sends it with a delivery representative.                  |
| Store Owner/Manager | Maintains inventory and manages the physical flow of goods: monitors low-stock alerts, views sales history, and generates optimized purchase orders for suppliers.                        |
| Investor            | Accesses a secure, read-only executive dashboard to evaluate business health, pulling aggregate reports on KPIs such as monthly GMV, acquisition cost, and retention rate.                |
| Delivery Worker     | Uses the mobile delivery app for final-mile logistics: receives an optimized route, scans a manifest to set orders to Out for Delivery, and marks each stop Delivered with digital proof. |
| Payment Service     | Interacts through a dedicated, secure API. On order submission, the system sends a transaction request with order details and a payment token.                                            |

## 4.2 Use Cases

| ID  | Use Case                                | Primary Actor(s)                     |
|-----|-----------------------------------------|--------------------------------------|
| UC1 | Manage Digital Storefront and Inventory | Store Owner/Manager                  |
| UC2 | Place Order and Initiate Transaction    | Customer                             |
| UC3 | Process Payment                         | Payment Service                      |
| UC4 | Control Order Fulfillment Cycle         | Store Owner/Manager, Delivery Worker |
| UC5 | Monitor Business Performance            | Investor                             |

## 5. Functional Model

### 5.1 Main System Scenario: Seamless Purchase

Amal, the customer, opens the Cloud Store app and logs in. She browses the categorized inventory, adds several items to her cart, and confirms her delivery address and window. When she submits the order, the system securely communicates with the Payment Service to confirm the transaction. Once paid, the system alerts the store manager, who generates an optimized picking list and reserves the stock. The order is then assigned to Faris, the delivery worker, who receives the best route on his mobile app. When Faris picks up the order, Amal is notified that her groceries are Out for Delivery. On arrival, Faris confirms the delivery and captures Amal's digital signature, completing the journey from virtual shelf to front door.

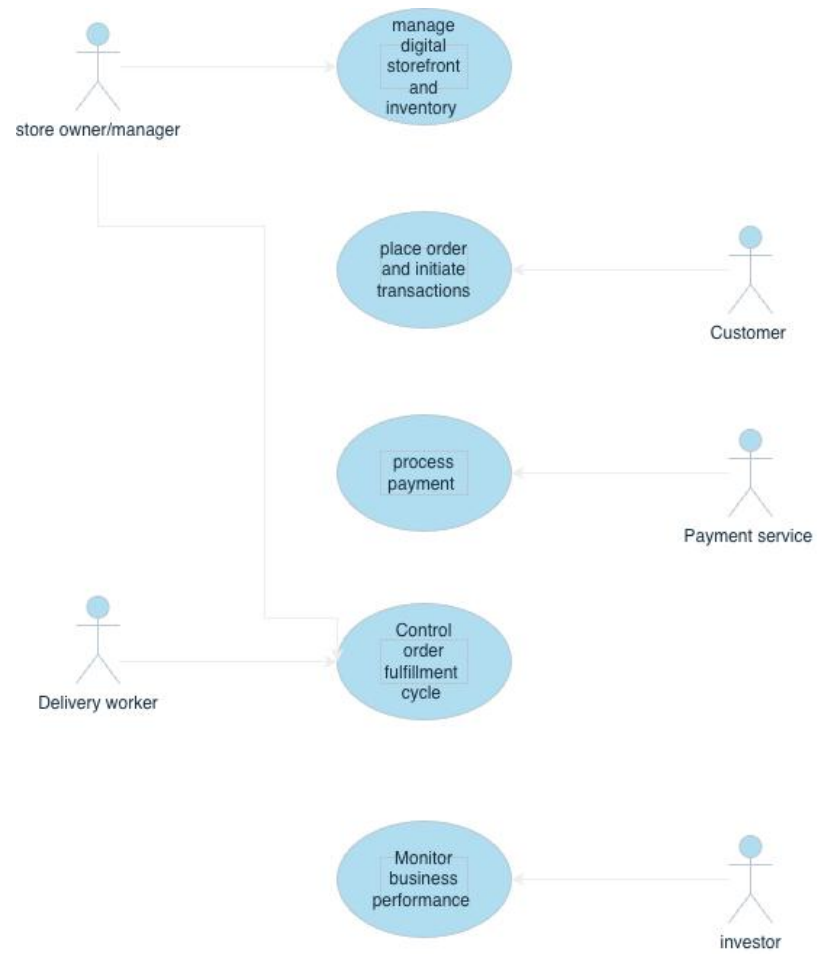


Figure 1: Use case model

## 5.2 Detailed Scenario: Complete Digital Grocery Purchase and Delivery

**Participating actor instances:** Amal (Customer), GlobalPay (Payment Service), Saleh (Store Owner/Manager), Faris (Delivery Worker).

### Flow of events

1. Amal opens the Cloud Store app and logs in. The system displays the Product Catalog View (UC2).
2. Amal browses categories, searches for items, and places 12 grocery items (including five units of Organic Chicken Breast) in her cart.
3. At checkout she confirms her saved home address and selects a two-hour delivery window for the next morning.
4. Amal selects a saved credit card and submits the order, initiating the transaction.
5. The system contacts GlobalPay through the Payment API with the transaction amount and card token.
6. GlobalPay processes the request, and the system receives a Payment Success confirmation within 2 seconds.
7. The system updates the order status to Awaiting Fulfillment and emails a confirmation to Amal.
8. Saleh sees a new batch of orders requiring fulfillment for the 9 AM slot on his Manager Dashboard.
9. Saleh clicks Fulfill Batch; the system generates a picking list optimized by warehouse layout and reserves the inventory (decrementing InventoryStock) for the 12 items.
10. Warehouse staff complete the picking, and Saleh marks the batch Ready for Pickup.
11. The system assigns the orders to Faris, whose location and schedule were considered by the RouteOptimizer. Faris is notified in his Delivery App.
12. Faris confirms the batch at the warehouse and sets the status to Out for Delivery. Amal receives a notification with a tracking map link.
13. Faris drives the optimized route and hands over the goods at Amal's address.
14. Faris captures Amal's digital signature as proof of delivery and marks the order Delivered.
15. The system sets the final status to Delivered, logs the event, and makes the sales data available for executive reports.

**Alternate flow:** if the Organic Chicken Breast is out of stock during fulfillment (step 9), the system alerts Saleh, who contacts Amal to offer a substitute or refund, then re-triggers picking with the adjusted item list.

## 6. Non-Functional Requirements

| Quality       | Requirement                                                                                                                                                              |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Security      | All PII and transaction data related to UC2/UC3 must be handled in strict compliance with PCI DSS and secured with up-to-date encryption protocols.                      |
| Integrability | The Payment Service API (UC3) and the Delivery App API (UC4) must be fully documented using a formal specification (e.g. OpenAPI) to ensure stability and extensibility. |
| Performance   | The entire transaction pipeline, from order submission (UC2) through payment confirmation (UC3), must complete in under 3 seconds.                                       |
| Availability  | The storefront (UC2) and payment integration (UC3) must maintain 99.9% uptime to guarantee continuous sales capability.                                                  |
| Scalability   | The system must handle a 500% increase in transaction volume over 12 months without architectural redesign or degraded response time.                                    |
| Auditability  | All inventory and financial data must be logged, traceable, and available for automated report generation (UC5), with data retention of at least seven years.            |

## 7. Analysis Object Model

| Category | Purpose                                                                                                      | Objects                                                                                                                    |
|----------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Boundary | Interface with the outside world (actors): screens, forms, reports, and external APIs.                       | ShoppingCartView, ProductCatalogView, PaymentAPI, DeliveryAppInterface, LoginScreen, ManagerDashboard, ExecutiveReportView |
| Entity   | Model persistent information and core domain concepts; long-lived and represent the data.                    | Customer, Product, Cart, Order, LineItem, InventoryStock, DeliveryRoute, Address, PaymentToken                             |
| Control  | Manage application logic for specific use cases, orchestrating the flow between boundary and entity objects. | OrderPlacementHandler, PaymentProcessor, FulfillmentManager, RouteOptimizer, ReportGenerator                               |

7.1 Class Diagram

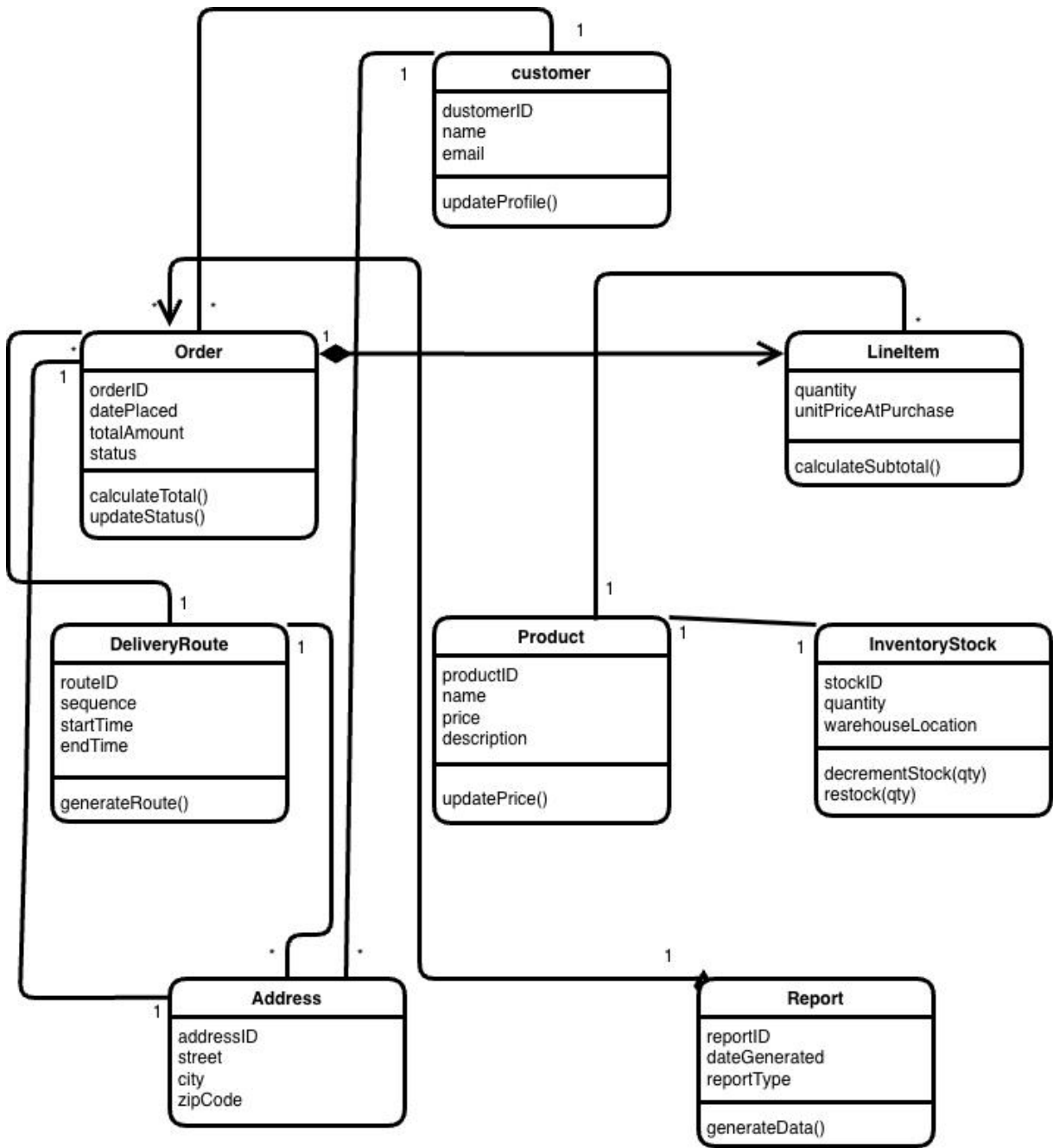


Figure 2: Class diagram of the core domain

## 8. Dynamic Model

### 8.1 Sequence Diagram: Place Order and Reserve Stock (UC2)

The customer presses checkout; the OrderPlacementHandler loops over the cart to check availability and reserve stock in InventoryStock, then creates the Order with status PENDING\_PAYMENT and returns the order ID for confirmation.

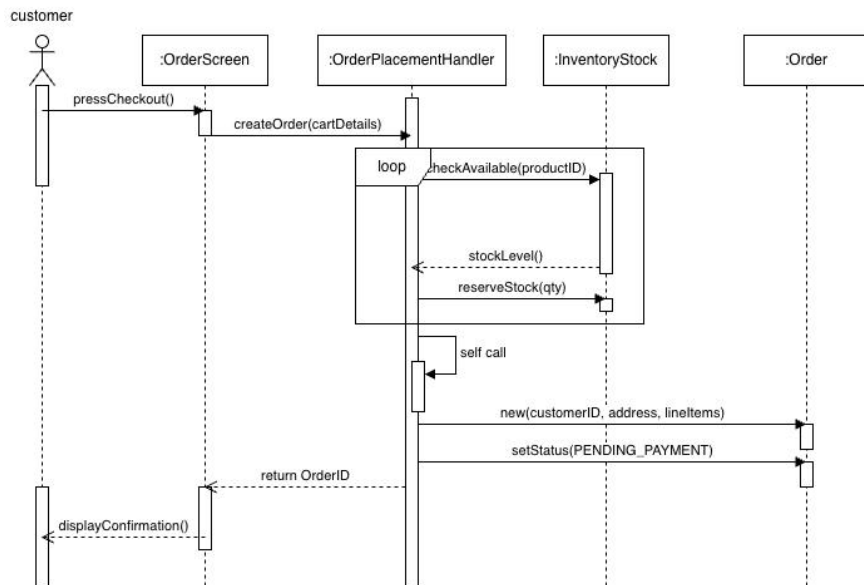


Figure 3: Sequence diagram: place order and reserve stock

### 8.2 Sequence Diagram: Control Order Fulfillment Cycle (UC4)

The store manager initiates fulfillment for a batch of orders; the FulfillmentManager creates a DeliveryRoute that generates an optimized sequence, updates each Order to OUT\_FOR\_DELIVERY, notifies the delivery worker, and returns the route details for confirmation.

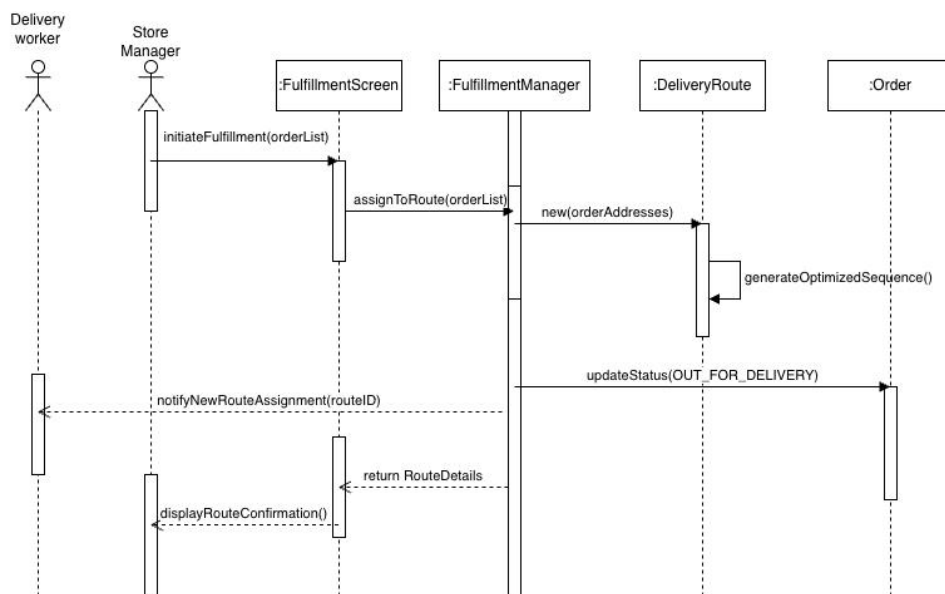


Figure 4: Sequence diagram: control order fulfillment cycle

### 8.3 State Machine: FulfillmentManager Control Object

The FulfillmentManager moves from Awaiting Batch Selection through Route Generation, Assignment & Notification, and Delivery Monitoring, then archives each order as delivered until all orders are archived.

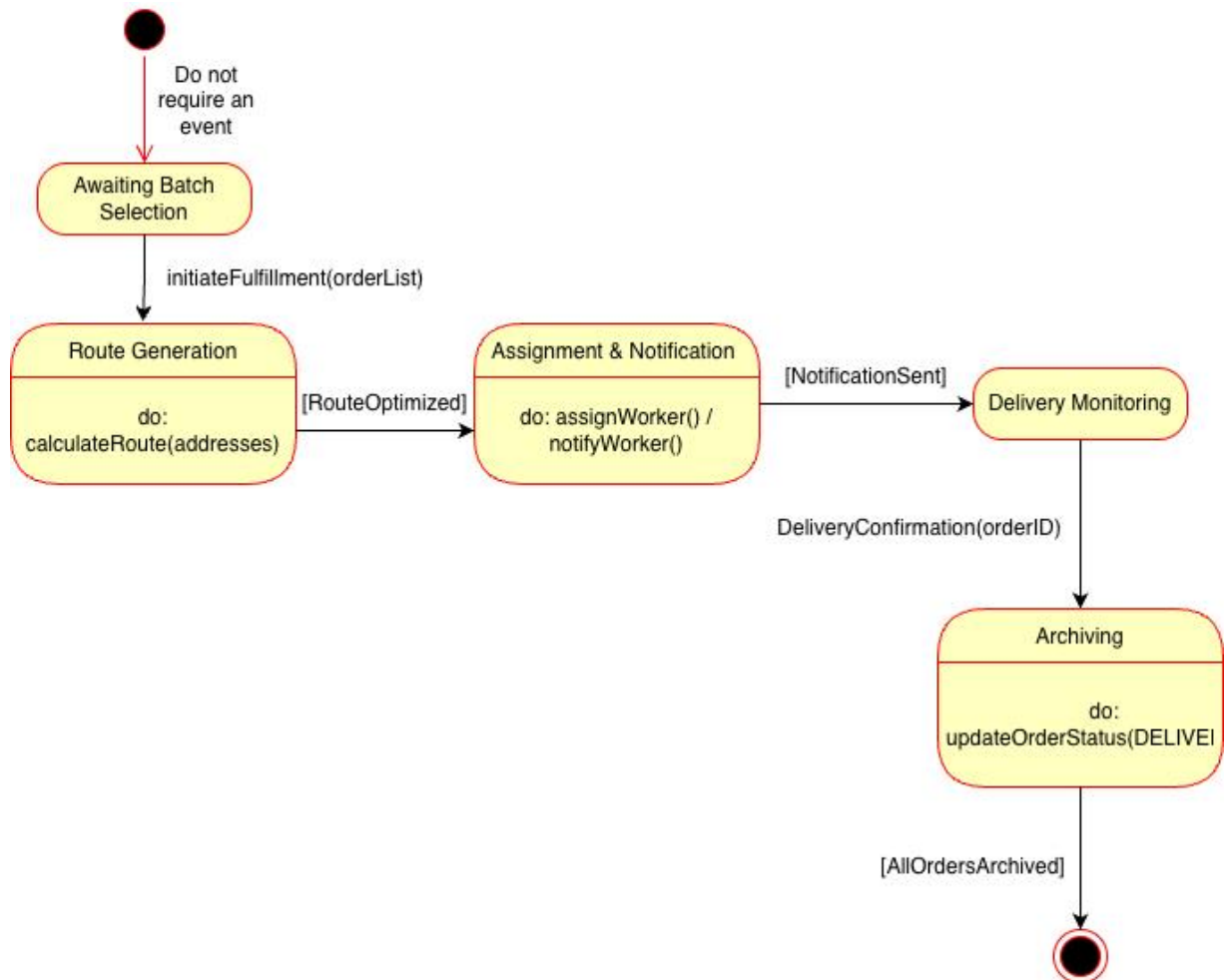


Figure 5: State machine diagram for the FulfillmentManager

## 9. Business Process Models

### 9.1 Order Processing and Fulfillment

Covers the flow across the customer, the Cloud Store system, and the logistics team: payment validation, a stock-availability gateway (with an out-of-stock notification path), stock reservation, route scheduling, picking and delivery, and delivery confirmation.

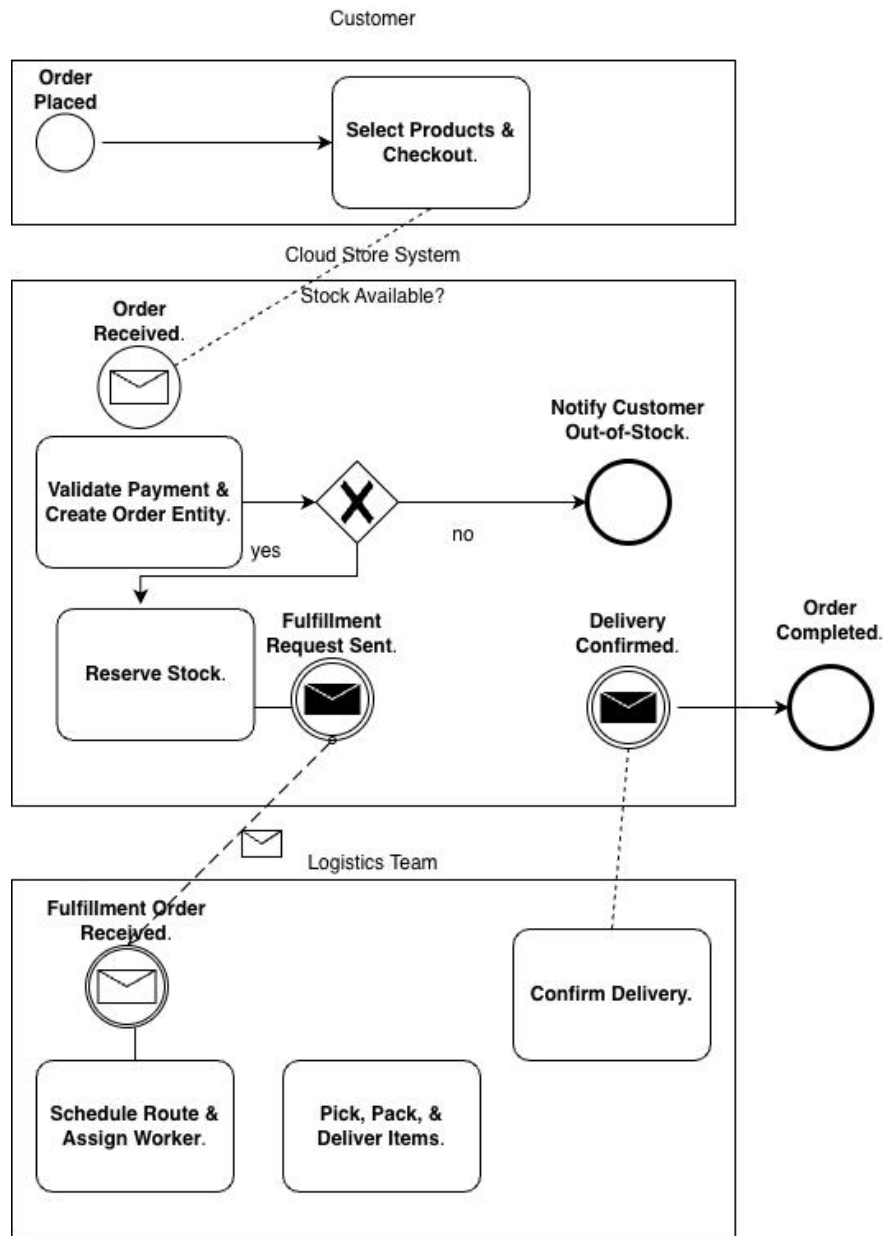


Figure 6: BPMN: order processing and fulfillment

9.2 Inventory Restock Management

Covers the flow across the Cloud Store system, the store manager, and the supplier: a reorder alert when stock falls below the reorder point, purchase order creation and sending, supplier shipment, goods inspection, and the InventoryStock update.

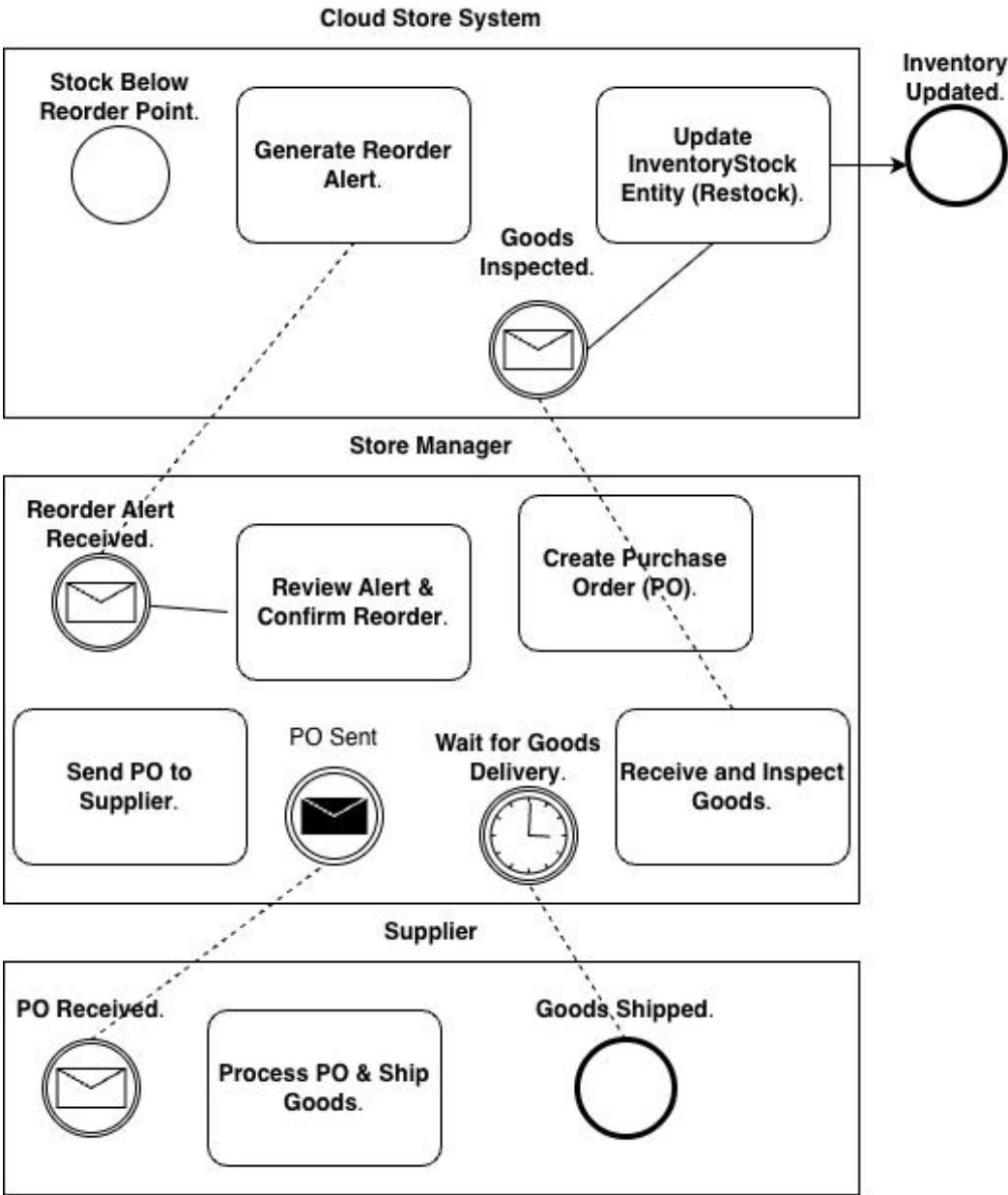


Figure 7: BPMN: inventory restock management